Lab. 1 – Introduzione a Verilog

Laboratorio di Reti Logiche e Calcolatori Elettronici

Michele Lora

Contatti e ricevimento

Michele Lora
email: `michele.lora@univr.it`
Stanza 1.69 – Ca' Vignal 2

Ricevimento:

Martedí 10.00 – 12.30[a], previo **appuntamento** seguendo le indicazioni riportate sulla **pagina istituzionale del docente**:

- Richiedere l'appuntamento via email istituzionale[b] con almeno due giorni lavorativi di anticipo rispetto alla data desiderata.
- È possibile richiedere appuntamenti in orari diversi da quello ufficiale.
- Il ricevimento è sospeso nei due giorni precendeti a scadenze relative a prove di esame.

[a] L'orario di ricevimento è indicativo e soggetto a variazioni in base agli impegni del docente.
[b] Email da domini diversi di `@studenti.univr.it` verranno automaticamente ignorate.

Struttura del corso

Il corso affronta la progettazione di un dispositivo di calcolo elettronico:

- Nel primo semestre si affronta la progettazione di dispositivi hardware;
- Nel secondo semestre si affronta lo sviluppo di software che controlla l'hardware.

Primo semestre

1. Introduzione a Verilog
2. Circuiti combinatori
3. Circuiti sequenziali
4. Macchine a stati finiti e FSMD
5. Modellazione gerarchica
6. Sintesi automatica

Secondo semestre

1. Introduzione ad Assembly
2. Sintassi di Assembly
3. Debugger, numeri e stringhe, makefile
4. Chiamate a funzione e stack
5. Assembly Inline in C
6. RISC-V: Verilog e Assembly

Modalità d'esame

La valutazione del laboratorio avviene attraverso la presentazione di due progetti:

1. **Progetto Verilog**: progettazione di un dispositivo hardware, composto da più sottocomponenti, sintesi logica del circuito.
2. **Progetto Assembly**: sviluppo di un software in Assembly per architetture Intel x86.

I progetti vanno consegnati a scadenze precise[1]. E vanno svolti in gruppo:

- Gruppi composti da minimo due e massimo tre persone;
- I gruppi composti da tre persone dovranno svolgere una parte aggiuntiva[2];
- I componenti del gruppo possono variare tra i due progetti;
- I componenti del gruppo possono essere qualsiasi studente, inscritto a qualsiasi anno di corso, che non abbia ancora completato l'esame di Reti Logiche e Calcolatori Elettronici o il corso di Architettura degli Elaboratori (CdL in Informatica).

[1] Sono previste modalità di presentazione dei progetti fuori dalle sessioni d'esame.
[2] la parte aggiuntiva richiede una quantità di lavoro inferiore al 50% di quanto richiesto dal progetto.

Calcolo del punteggio

Una volta consegnato un progetto, questo viene controllato. I progetti che passano i controlli vengono ammessi all'orale, a cui si dovrà presentare l'intero gruppo.
Il punteggio di ciascun progetto viene assegnato all'orale. Il voto di laboratorio viene calcolato come segue:

- Ogni progetto vale da 0 a 3 punti;
- Il voto di laboratorio è dato dalla media dei punteggi ottenuti nei due progetti. Il voto viene sommato al voto ottenuto nella parte di Teoria.
  - **Corollario**: il punteggio 0 indica che il progetto è sufficiente e può essere
- Consegnando il progetto alla prima data utile, si potrà ottenere un punto aggiuntivo.
  - Progetti Verilog consegnati a febbraio, progetti assembly consegnati a giugno.
- I punteggi verranno assegnati in base alla qualità dell'esame orale
  - L'esame orale consiste nella presentazione del progetto, alcune domande sul progetto stesso e alcune domande su argomenti del progetto collegati ad argomenti teorici;
  - I diversi componenti dello stesso gruppo possono ottenere punteggi diversi, in base alla loro preparazione. Tuttavia, dovranno essere tutti sufficienti o non lo sarà nessuno.

Calcolo del punteggio

Decremento del voto di Teoria

Nel caso in cui uno studente completi il modulo di Teoria in una determinata sessione d'esame, ma non abbia completato entrambi il modulo di Laboratorio entro la fine della stessa sessione, il voto di Teoria potrèssere portato alla sessione successiva decrementato del 15%.
Non sono previsti, in nessun caso, decrementi del punteggio ottenuto nel modulo di Laboratorio.

Esempio 1: Uno studente supera le due prove parziali, completando il modulo di Teoria nella sessione Estiva (giugno - luglio) 2026 con un voto di 30. Lo stesso studente consegna e discute il primo elaborato nella sessione autunnale (settembre) 2026 con un punteggio di 3 punti, e il secondo elaborato nella sessione invernale (febbraio) 2027 con un punteggio di 3 punti.

- Lo studente subirà un primo decremento del voto di Teoria per portare il voto dalla sessione estiva a quella autunnale: $30 * 0.85 = 25.5$.
- non avendo ancora completato gli elaborati, subirà un ulteriore decremento per portare il voto di Teoria dalla sessione autunnale a quella estiva: $25.5 * 0.85 = 21.68$.
- Una volta presentato con successo il secondo elaborato, il punteggio di laboratorio (3 punti) viene sommato al voto di Teoria: $21.68 + 3 = 24.68$.
- Il punteggio viene arrotondato a 25 e verbalizzato nel libretto dopo la sessione di febbraio 2027.

Calcolo del punteggio

Esempio 2: Uno studente non supera la Teoria, ma consegna entrambi gli elaborati nella sessione estive (Giugno - Luglio), ottenendo un punteggio di 3 punti negli elaborati. Successivamente, supera la prova di Teoria a settembre con il punteggio di 22.

- Il voto di Teoria non subirà nessun decremento.
- Nella sessione autunnale, lo studente potrà verbalizzare il voto ottenuto: $22 + 3 = 25$

Attenzione!

I progetti di Laboratorio richiedono di applicare quanto imparato nella Teoria.
La grande maggioranza degli studenti che non superano le prove di Laboratorio non hanno ancora preparato la parte di Teoria.

Consiglio: molti concetti visti nella Teoria consentono di semplificare notevolmente lo svolgimenti degli elaborati, aumentando significativamente le possibilità di superare gli orali di laboratorio con il punteggio massimo.

Linguaggi di descrizione del hardware

- **Hardware Description Language** (HDL): linguaggio "di programmazione" utilizzato per descrivere il comportamento e la struttura di sistemi hardware.
  - In realtà non sono veramente linguaggi di programmazione, in quanto non vengono utilizzato per creare programmi software.
- I principali HDL sono, storicamente, **Verilog** e **VHDL**. Più di recente sono emersi nuovi linguaggi, come **SystemVerilog**, **Bluespec**, **Chisel**, **MyHDL**, etc.
- Verilog (e la sua estensione SystemVerilog) è il linguaggio di descrizione dell'hardware più utilizzato nell'industria.
- Verilog è un linguaggio di descrizione dell'hardware **event-driven**, ossia il comportamento del sistema è descritto mediante la definizione di eventi che scatenano delle azioni.

Modellazione Hardware $\neq$ Programmazione

Un linguaggio di descrizione dell'hardware **NON É** un linguaggio di programmazione

- **Linguaggio di programmazione**: linguaggio che viene tradotto in istruzioni macchina che vengono eseguite da un processore;
- **Linguaggio di descrizione dell'hardware**: linguaggio che permette di **modellare** il comportamento temporale e la struttura spaziale di un circuito elettronico.
  - Descrive un oggetto fisico che si vuole progettare, mentre nella programmazione, un software é esso stesso l'oggetto;
  - l'esecuzione é intrinsecamente **concorrente**, in quanto la fisica dei componenti hardware impone che più fatti avvengano nello stesso istante.

Livelli di astrazione in Verilog

- **Comportamentale**: il comportamento del sistema hardware viene descritto specificando le azioni da comprendere, in maniera quasi algoritmica.
- **Strutturale** o **Dataflow**: il comportamento del sistema hardware viene descritto come aggregazione e interconnessione di componenti, i quali a loro volta specificheranno delle funzionalità in uno dei tre stili di modellazione.
- **Gate-level**: il comportamento del sistema hardware viene descritto come aggregazione di porte logiche di base, quali porte **and**, **or**, **xor**, **nand**, etc.

Tipi di dato

I segnali in Verilog possono avere solo i quattro valori corrispondenti a stati fisici di segnali hardware:

- `'0'`: zero logico, oppure una condizione false;
- `'1'`: uno logico, oppure una condizione vera;
- `'X'`: valore sconosciuto, oppure don't care;
- `'Z'`: alta impedenza del segnale.

È possibile esprimere valori numerici a più bit utilizzando i letterali numerici, la cui struttura base è: `D'Bvalore`

- `D`: dimensione del segnale (numero di bit) espresso come numero in base 10.
- `B`: base, può essere una delle seguenti: `d`: base 10, `b`: base 2, `o`: base 8, `h`: base 16 (esadecimale).
- Esempi: `32'd17` $= (17)_{10}$, `32'hABCD` $= (43981)_{10}$

Tipi di dato

Gli elementi di memorizzazione e le connessioni tra componenti vengono rappresentati mediante due costrutti principali:

- `wire`: rappresenta la connessione tra elementi, il cui valore è guidato solamente dall'uscita di una rete logica (oppure, mediante **assegnamenti continui**).
- `reg`: modella il comportamento di un elemento di memorizzazione.

Per specificare **modelli a livello comportamentale** è possibile utilizzare variabili, simili a quelle dei linguaggi di programmazione:

```
integer i; // variabile intera a 32 bit.
real r;    // variabile Reale (floating point).
time t;    // variabile di tipo tempo.
```

Le variabili non possono essere dichiarate dentro a blocchi sequenziali, ma solo nel preambolo del modulo.

Moduli e interfacce

Il costrutto `module` è l'unità fondamentale di un modello Verilog.

Un modulo rappresenta un componente del sistema, caratterizzato da un'**interfaccia**, ossia la definizione delle sue **porte di ingresso** e **porte di uscita**.

```
1 module complemento(input [3:0] in, output [3:0] out);
2 begin
3 ...
4 endmodule
```

Modellazione a Behavioral Level

Verilog permette di specificare il comportamento di circuiti elettronici descrivendo in modo **algoritmico** il comportamento del circuito.
É possibile descrivere il comportamento del circuito definendo **come evolvono i segnali**, quali **operazioni logiche e aritmetiche** vengono eseguite sui segnali, definendo variabili e costanti, ed utilizzando altri costrutti tipici dei linguaggi di programmazione.
Il linguaggio Verilog mette a disposizione vari costrutti condizionali ed iterativi:

- costrutto condizionale `if`;
- costrutto condizionale `case`;
- costrutto di ciclo `while`;
- costrutto di ciclo `for`;
- blocco sequenziale `repeat`.

Blocchi sequenziali

Blocco sequenziale: costrutto principale per la modellazione del comportamento di un sistema elettronico. È descritto da un insieme di istruzioni comprese tra le parole `begin` and `end`. Esistono due tipi di blocchi sequenziali:

- blocco `always`: blocco che viene eseguito ogniqualvolta si verifica un evento.
- blocco `initial`: blocco che descrive il comporamento del sistema a partire dall'istante iniziale della sua esecuzione.

Sintassi always

```
always @( evento )
begin : nome
    ...
end
```

Sintassi initial

```
initial begin : nome
    ...
end
```

Esempio: Sommatore a 4 bit

```
module sommatore_4_bit(
    input [3:0] in1,
    input [3:0] in2,
    output reg [3:0] out,
    output reg cout
    );


    always @(in1, in2) begin
        out <= in1 + in2;
        if(in1 + in2 > 15)
            cout <= 1;
        else
            cout <= 0;
    end

endmodule
```

Simulazione

la simulazione è il **processo di emulazione** (virtuale) del comportamento di un circuito elettronico per **verificarne il comportamento**, senza costruire fisicamente il circuito. Permette di osservare come il circuito risponderebbe a vari segnali e condizioni, rilevando eventuali errori o miglioramenti, prima di passare alla realizzazione fisica hardware.

- L'esecuzione di un modello Verilog é la simulazione del circuito descritto nel modello, e richiede:
  - **Simulatore**: software che esegue la simulazione di un modello Verilog;
  - **Testbench**: modello Verilog che fornisce i segnali di ingresso al circuito da simulare.
  - **Forme d'onda**: rappresentazione grafica dei segnali, e sono necessari strumenti per la lettura delle forme d'onda.

Stimolare il modello: testbench

Il **testbench** è il modulo che genera gli ingressi e legge le uscite del modello che si sta progettando. Dunque, il suo ruolo è quello di stimolare l'esecuzione del modello in esame. Due alternative principali:

- **module** ad-hoc che istanzia il componente in esame;
- **script** di comandi che generano gli ingresso del componente in esame.

In questo corso, ci limitiamo a utilizzare la prima soluzione. Comandi utili alla creazione di un testbench:

- `$dumpfile(nomefile.vcd)`: crea un file di tracce (waveform) in cui salvare l'andamento della simulazione;
- `$dumpvars(num, modulo)`: traccia le variabili a partire dal modulo indicato. Se `num` é 0, traccia tutte le variabili del modulo e dei suoi sotto-moduli, se 1, solo quelle del modulo indicato;
- `$display(argomenti)`: stampa a video la stringa creata specificando gli argomenti (sintassi C-like).

Testbench: esempio, sommatore a 4 bit

```
module testbench_4_bit();
    reg [3:0] a, b;
    wire [3:0] out;
    wire c;
    sommatore_4_bit a4(.in1(a), .in2(b), .out(out), .cout(c));
    initial begin
        $dumpfile("test_sommatore4.vcd");
        $dumpvars(0, testbench_4_bit);
        a <= 4'd2;
        b <= 4'd1;
        #5
        $display("%d + %d = %d, riporto di %d", a, b, out, c);
        a <= 4'd3;
        b <= 4'd2;
        #5
        $display("%d + %d = %d, riporto di %d", a, b, out, c);
        $finish;
    end
endmodule
```

Simulazione di Verilog: Verilator

- **Verilator** è un compilatore e simulatore open-source per il linguaggio Verilog.
- Supporta la sintassi Verilog 2001, 2005 e una parte di SystemVerilog 2005.
- **Installazione**:
  - Su Linux: installazione attraverso il gestore di pacchetti, **e.g.**, per le distribuzioni basate su Debian: `sudo apt-get install verilator`;
  - Su Windows: disponibile utilizzando il Windows Subsystem for Linux (WSL2), Cygwin o MinGW;
  - Su MacOS: installazione attraverso Homebrew, **i.e.** `brew install verilator`.

Simulazione con Verilator

- Compilare il codice Verilog:

```
verilator timing --trace --binary <lista dei file verilog> \
        --top-module <modulo principale> --Mdir <cartella> \
        -o <eseguibile>
```

  - `--trace`: abilita la creazione delle tracce;
  - `--binary`: compila i sorgenti nel file binario;
  - `--top-module`: permette di specificare il nome del modulo principale (top-level: il componente che contiene tutti gli altri componenti).
  - `--Mdir`: permette di specificare la cartella di output dei file di lavoro. Il parametro è facoltativo, ma consigliato.
  - `-o`: specifica il nome del file eseguibile che verrà creato nella cartella di lavoro.
  - **lista dei file verilog**: deve contenere tutti i file, il primo della lista deve specificare una risoluzione temporale.
- Eseguire la simulazione:

```
<cartella>/<eseguibile>
```

Esempio: sommatore a 4 bit

Scaricare da Moodle il file esempi.tar.gz delle prima lezione nella cartella Scaricati.

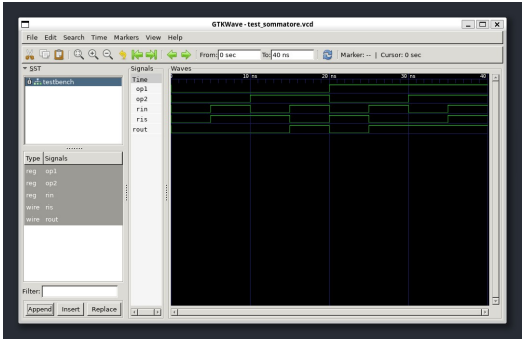
```
$> mkdir RLCE
$> mkdir RLCE/Lezione1
$> mv Scaricati/esempi.tar.gz RLCE/Lezione1
$> cd RLCE/Lezione1/
$> tar xzf esempi.tar.gz
$> cd esempi/sommatore

$> verilator --trace --binary testbench_4_bit.v sommatore_4_bit.v \
    --top-module testbench -o somma4
$> ./obj_dir/somma4
```

Nota: non avendo specificato l'opzione `--Mdir` è stata utilizzata l'opzione di default `obj_dir`.

Visualizzazione delle forme d'onda

GTKWave è un visualizzatore di forme d'onda open-source. Permette di visualizzare il comportamento dei segnali durante una simulazione del circuito.

- **Installazione**:
  - Linux: gestore di pacchetti
    e.g., sudo `apt-get install gtkwave`
  - Windows: scaricare il pacchetto di installazione dal sito ufficiale.
  - MacOS: `brew install gtkwave`
- **Esecuzione**:
  - Aprire il file di tracce generato da Icarus Verilog:
    `$> gtkwave nomefile.vcd`
- Permette di visualizzare i valori dei segnali in diverse basi (decimale, esadecimale, ottale, binaria).



Esempio: complemento a due

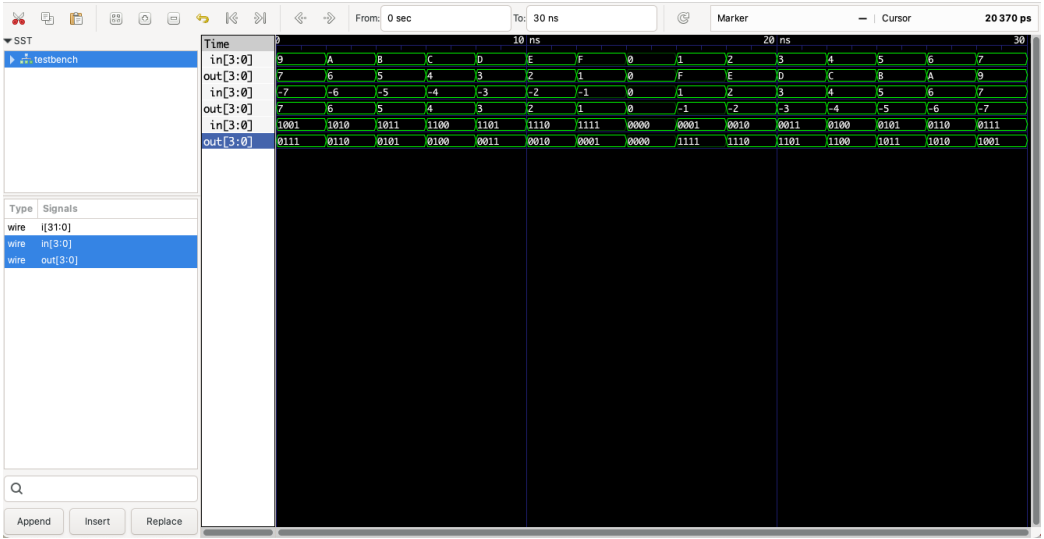
```
`timescale 1ns / 1ps
module testbench();
    reg [3:0] in;
    wire [3:0] out;
    integer i;
    complemento c(.in(in), .out(out));
    initial begin
        $dumpfile("simulazione.vcd");
        $dumpvars(0, testbench);
        for(i = -7; i <= 7; i = i + 1) begin
            $display("Prossimo input: %d", i);
            in = i[3:0];
            #2;
            $display("Output: %d", out);
        end
    end
endmodule
```

Esempio: complemento a due

```
1  module complemento(input [3:0] in,
       output [3:0] out);
2      integer i;
3      reg [3:0] negato;
4      always @(in)
5      begin
6          for(i = 0; i < 4; i = i + 1)
               begin
7              negato[i] <= !in[i];
8          end
9      end
10     assign out = negato + 1;
11 endmodule
```

```
$> cd complemento
$> verilator --trace --binary tb.v
    complemento.v --top-module
    testbench -o complemento
$> obj_dir/complemento
$> gtkwave simulazione.vcd
```

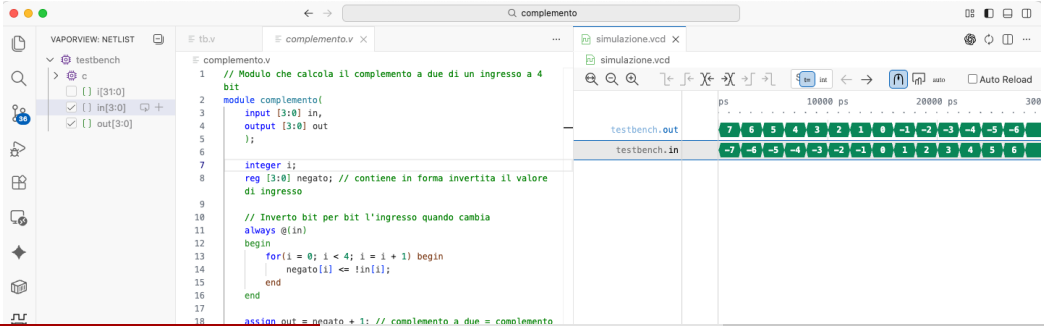
Esempio: complemento a due



Strumenti consigliati

Editor consigliato: Visual Studio Code

- Strumento open-source e multipiattaforma
- Diverse estensioni utili: Verilog Syntax extensions and trace extensions
  - Verilog syntax extension: `https://marketplace.visualstudio.com/items?itemName=mshr-h.VerilogHDL`
  - Integrated traces viewer (VaporView): `https://marketplace.visualstudio.com/items?itemName=lramseyer.vaporview`



Esercizi

1. Scrivere un componente che riceve in ingresso due numeri a 4 bit ed un segnale di controllo. Il componente ha un'uscita a 4 bit. Se il segnale di controllo vale '0' il componente effettuerà la somma dei due numeri in ingresso; altrimenti, eseguirá la sottrazione.
2. Scrivere il testbench per il componente progettato nell'Esercizio 1. Il testbench dovrà stampare a video gli ingressi passati al componente e i valori di output generati.
3. Verificare il comportamento del modulo e del suo testbench attraverso l'utilizzo di GTKWave.